

Homework 2: DDPG, PPO, and TRPO

Submission Guidelines: Your deliverables shall consist of 2 separate files – (i) A PDF file: Please compile all your write-ups into one .pdf file (photos/scanned copies are acceptable; please make sure that the electronic files are of good quality and reader-friendly); (ii) A zip file: Please compress all your source code into one .zip file. Please submit your deliverables via E3.

Problem 1 (PPO With a Clipped Objective)

(10 points)

In Lecture 15, we introduce PPO-Clip, which updates the policy iteratively by maximizing the following objective function:

$$L^{\text{clip}}(\theta; \theta_k) := \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta_k}}, a \sim \pi_{\theta_k}(\cdot | s)} [L_{s,a}^{\text{clip}}], \quad (1)$$

$$\text{where } L_{s,a}^{\text{clip}}(\theta; \theta_k) := \min \left\{ \frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)} A^{\pi_{\theta_k}(s, a)}, \text{clip} \left(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}(s, a)} \right\}. \quad (2)$$

As mentioned in class, another possibility is to consider the following variant:

$$\tilde{L}_{s,a}^{\text{clip}}(\theta; \theta_k) := \text{clip} \left(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)}, 1 - \epsilon, 1 + \epsilon \right) \cdot A^{\theta_k}(s, a). \quad (3)$$

Could you explain the behavioral differences between the two objective functions $L_{s,a}^{\text{clip}}(\theta; \theta_k)$ and $\tilde{L}_{s,a}^{\text{clip}}(\theta; \theta_k)$? (Hint: For a clear comparison, it could be better to make a table and illustrative plots similar to Figure 1 below)

	$p_t(\theta) > 0$	A_t	Return Value of $\min$	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$	+	$p_t(\theta) A_t$	no	+	✓
2	$p_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$	−	$p_t(\theta) A_t$	no	−	✓
3	$p_t(\theta) < 1 - \epsilon$	+	$p_t(\theta) A_t$	no	+	✓
4	$p_t(\theta) < 1 - \epsilon$	−	$(1 - \epsilon) A_t$	yes	−	0
5	$p_t(\theta) > 1 + \epsilon$	+	$(1 + \epsilon) A_t$	yes	+	0
6	$p_t(\theta) > 1 + \epsilon$	−	$p_t(\theta) A_t$	no	−	✓

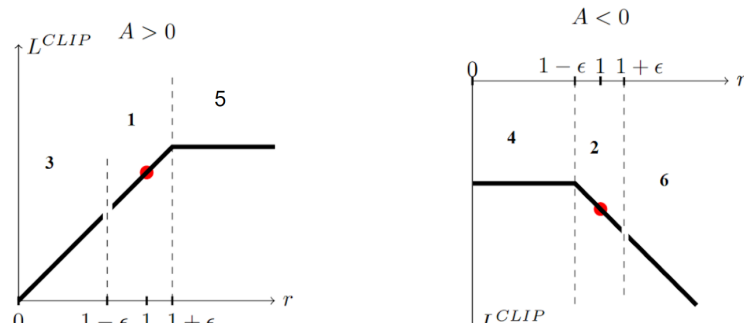


Figure 1: Behavior of the original PPO-clip objective.

Problem 2 (Convergence of TD(0) and Stochastic Approximation)

(2+3+10+5=20 points)

In Lecture 12, we learned the fundamental concept of stochastic approximation (SA) to analyze the convergence of temporal difference methods. Specifically, we leveraged the extended SA theorem by (Jaakkola, Jordan,

and Singh, 1994) available at <https://web.eecs.umich.edu/~baveja/Papers/Neural-Computation-94.pdf> to introduce the asymptotic convergence of TD(0), which learns V^π by the following update: At each time t , suppose we observe a transition $(s_t, a_t, r_{t+1}, s_{t+1})$ (possibly from a generative model). Then, TD(0) algorithm updates the estimated value function by

$$V_{t+1}(s_t) = V_t(s_t) + \alpha_t(s_t)(r_{t+1} + \gamma V_t(s_{t+1}) - V_t(s_t)). \quad (4)$$

In this homework problem, you will have a nice opportunity to formally apply SA to show the convergence. Recall that for a stochastic process $\{W_t\}$ (where $W_t \in \mathbb{R}^{|S|}$ and $|S|$ is a finite set) constructed as

$$W_{t+1}(s) = (1 - \alpha_t(s))W_t(s) + \alpha_t(s)\varepsilon_t(s), \quad \forall s \in \mathcal{S}, \quad (5)$$

then we have the following theorem. Let $\mathcal{H}_t$ denote the history of all observations up to time t .

Theorem 1: Stochastic Approximation

If the following conditions hold:

- (C1) $\sum_{t \geq 0} \alpha_t(s) = \infty$ and $\sum_{t \geq 0} \alpha_t(s)^2 < \infty$, with probability one.
- (C2) $|\mathbb{E}[\varepsilon_t(s)|\mathcal{H}_t]| \leq \rho \|W_t\|_\infty$, where $\rho \in (0, 1)$.
- (C3) $\mathbb{V}[\varepsilon_t|\mathcal{H}_t] \leq C(1 + \|W_t\|_\infty)^2$, where C is some finite constant.

Then, we have $W_t \rightarrow 0$ as $t \rightarrow \infty$, with probability one.

(a) Under the TD(0) algorithm, at each time step t , the value estimate of s_t is updated by Equation (4). Can you explicitly write down the update of $V_t(s)$ for those $s \neq s_t$? In this case, what is the $\alpha_t(s)$ for $s \neq s_t$?

(b) Under the TD(0) algorithm, what are $W_t(s_t)$ and $\varepsilon_t(s_t)$? Moreover, what are $W_t(s)$ and $\varepsilon_t(s)$ for $s \neq s_t$?

(c) Based on (b), please show that the conditions (C2) and (C3) indeed hold under the TD(0) algorithm.

(d) Suppose the condition (C1) holds. Please put everything together and show that $V_t(s) \rightarrow V^\pi(s)$ as $t \rightarrow \infty$, for every state $s \in \mathcal{S}$, with probability one.

Problem 3 (Solving TRPO Under Approximation Using Duality)

(10+10=20 points)

Recall from the slides of Lecture 8: We would like to solve the following optimization problem (denoted by (OPT)):

$$\text{Minimize}_{\theta \in \mathbb{R}^d} \quad -(\nabla_\theta L_{\theta_k}(\theta)|_{\theta=\theta_k})^\top (\theta - \theta_k) \quad (6)$$

$$\text{subject to} \quad \frac{1}{2}(\theta - \theta_k)^\top H_{\theta_k}(\theta - \theta_k) - \delta \leq 0. \quad (7)$$

We use θ^* to denote an optimizer of the above *primal* optimization problem (6)-(7). Note that in the above we write “Minimize” instead of “Maximize” simply to follow the conventions of the literature of optimization theory. Here we focus on the case where H is a *positive definite* matrix to avoid the technicalities (while H is only *non-negative definite* in general).

Based on the optimization theory, (OPT) is a convex optimization problem as both the objective and the constraints are convex functions. In this case, one standard way is to convert the constrained (OPT) into an

unconstrained *dual* problem by defining the *Lagrangian* $\mathcal{L}(\theta, \lambda)$ and the *dual function* $D(\lambda)$ as:

$$\mathcal{L}(\theta, \lambda) := -(\nabla_{\theta} L_{\theta_k}(\theta)|_{\theta=\theta_k})^{\top} (\theta - \theta_k) + \lambda \left(\frac{1}{2} (\theta - \theta_k)^{\top} H_{\theta_k} (\theta - \theta_k) - \delta \right) \quad (8)$$

$$D(\lambda) := \min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta, \lambda), \quad (9)$$

where λ is called the *Lagrange multiplier*. Moreover, we call the following the *dual problem* of (OPT):

$$\max_{\lambda \geq 0} D(\lambda). \quad (10)$$

For ease of notation, we define $\lambda^* := \arg \max_{\lambda \geq 0} D(\lambda)$ as the optimizer of the dual problem. By the standard theory of convex optimization, if there exists one strictly feasible point in (OPT), then the optimal value of the dual problem is equal to the original problem (usually called “strong duality”). Moreover, if strong duality holds and a dual optimal solution λ^* exists, then any optimizer of the primal problem is also a minimizer of $\mathcal{L}(\theta, \lambda^*)$, i.e., $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta, \lambda^*)$. For more details on duality, please refer to Chapter 5 of https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.

(a) In this problem, please show that the dual function $D(\lambda)$ of (OPT) can be written as:

$$D(\lambda) = \frac{-1}{2\lambda} \left((\nabla_{\theta} L_{\theta_k}(\theta)|_{\theta=\theta_k})^{\top} H_{\theta_k}^{-1} (\nabla_{\theta} L_{\theta_k}(\theta)|_{\theta=\theta_k}) \right) - \lambda \delta. \quad (11)$$

Accordingly, please find out λ^* based on (11).

(b) By the λ^* found in (a) and the property $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta, \lambda^*)$, show that $\theta^* = \theta_k + \alpha H_{\theta_k}^{-1} \nabla_{\theta} L_{\theta_k}(\theta)|_{\theta=\theta_k}$. What is the step size α ?

Problem 4 (Deep Deterministic Policy Gradient for Continuous Control) (20+20+15=55 points)

In this problem, we will implement the deep deterministic policy gradient (DDPG) algorithm with the help of neural function approximators, as discussed in Lectures 13-14. You may write your code in either PyTorch or TensorFlow (though the sample code presumes PyTorch framework). Moreover, you are recommended to use Weight & Bias to keep track of the loss terms and other related quantities of your implementation. If you are a beginner in learning the deep learning framework, please refer to the following tutorials:

- PyTorch: <https://pytorch.org/tutorials/>
- Weight and Biases: <https://docs.wandb.ai/tutorials>

For the deliverables, please submit the following:

- Technical report: Please summarize all your experimental results in 1 single report (and please be brief)
- All your source code
- Your well-trained models (both the actor and critic networks) saved in either .pth files or .ckpt files
- Record a short video (3-5 minute long) to describe your design and the experimental results that you observe in the following Problem 4(b)-4(c).

(a) We start by solving the simple “Pendulum-v1” problem (https://gymnasium.farama.org/environments/classic_control/pendulum/) using the DDPG algorithm. Read through `ddpg.py` and then implement the member functions of the classes **Actor**, **Critic**, and **DDPG** as well as the function **train**. Please briefly summarize your results (including the snapshots of Weight & Bias record) in the report and document all the hyperparameters (e.g. learning rates, NN architecture, and batch size) of your experiments. (Note: Pendulum is a rather basic environment mostly for the purpose of sanity check. As a result, typically it would take no more than 300 episodes to reach a well-performing policy, say a score of at least -130)

(b) Based on your implementation for (a), please adapt your DDPG algorithm to solve the “HalfCheetah”

locomotion task in MuJoCo (https://gymnasium.farama.org/main/environments/mujoco/half_cheetah/). Save your code in another python file named **ddpg_cheetah.py**. Please add comments to your code whenever needed for better readability.

Train your Halfcheetah for 500k environment steps and reach an average evaluation score (over 20 evaluation episodes) of at least 5,000 within 500k steps. Again, briefly summarize your results in the report and document all the hyperparameters of your experiments. (Note: As HalfCheetah is a more challenging environment than Pendulum, it would take a bit more training time to reach a well-performing policy, and it might require some efforts to tune the hyperparameters, e.g., learning rates and the batch sizes. That being said, it is typically expected that Halfcheetah can achieve a score of 6,000-10,000 within 500k environment steps)

(c) Based on your code for (b), further enhance your DDPG algorithm with the technique of “Clipped Double Q” (CDQ) for the critic update as in TD3, i.e., the loss of each critic network Q_{w_1}, Q_{w_2} with the clipped double Q shall be

$$L(w_i; \mathcal{B}) := \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \left(r + \gamma \min_{i'=1,2} Q_{w_{i'}}(s', \pi_\theta(s')) + \varepsilon - Q_{w_i}(s, a) \right)^2,$$

where $\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2), -\varepsilon_{\max}, \varepsilon_{\max})$

to solve the “HalfCheetah” task in MuJoCo. Save your code in another file named **ddpg_cdq_cheetah.py**. Please add comments to your code whenever needed for better readability.

Again, train your Halfcheetah for 500k environment steps. What observations can you make from the added CDQ technique, compared to the vanilla DDPG in subproblem (b)? Please briefly summarize your results in the report and document all the hyperparameters of your experiments.